

Daily Date Slack Bot

Serverless AWS automation · Terraform · Python

A serverless automation that posts the current date and day of the week to a Slack channel every weekday morning — solving a gap in Slack's native tooling (Scheduled Messages and Workflow Builder can't compute a dynamic date).

Problem

A recurring Slack message was being sent manually every weekday: a bolded/italicized date + day-of-week (e.g. `*_2026-09-03 Thursday_*`). Slack's built-in scheduling tools can only send static, pre-written text, so the message had to be typed by hand each morning. The automation also couldn't be tied to any one person's account or credentials, since ownership could change with a role change or departure.

Solution

An AWS Lambda function, triggered on a schedule by Amazon EventBridge, computes today's date in the target timezone and posts it to Slack via an incoming webhook. Because it runs as infrastructure rather than under a personal identity, it keeps running regardless of who's on the team.

Architecture:

- **AWS Lambda (Python 3.12)** — formats the date using the stdlib `zoneinfo` module for correct, DST-safe timezone conversion, and posts it to Slack.
- **Amazon EventBridge** — cron-based trigger (`cron(0 23 ? * SUN-THU *)` UTC = 7:00 AM Monday–Friday local time), replacing the need for any always-on server.
- **AWS SSM Parameter Store (SecureString)** — stores the Slack webhook URL encrypted at rest; the Lambda reads and decrypts it at runtime instead of hardcoding a secret.
- **Terraform** — defines the Lambda, EventBridge rule, and permissions as code, reusing an existing shared IAM execution role rather than provisioning new IAM resources.
- **Unit tests** (Python `unittest`) covering message formatting and the webhook POST behavior, including failure handling.

Key decisions

- **Matched existing team conventions** rather than introducing a new pattern: the target repository already standardized on Lambda + Terraform + EventBridge for Slack automations, so an initially-considered GitHub Actions approach was dropped in favor of consistency with the rest of the codebase.
- **No secrets in code or version control** — the webhook URL lives only in SSM Parameter Store, decrypted at runtime with least-privilege IAM permissions.

- **Reused shared infrastructure** (IAM role, Lambda layer for dependencies) instead of duplicating resources per bot, keeping the addition lightweight.
- **Validated infrastructure changes safely:** ran `terraform init`, `validate`, and a partial `plan` locally before code review, without ever running `apply` outside of a reviewed deployment process.

Outcome

A fully automated, ownerless daily message that requires no manual input and no dependency on any individual's Slack session or credentials — deployed through the team's standard infrastructure-as-code and pull-request review process.

Stack

AWS Lambda

Amazon EventBridge

AWS SSM Parameter Store

Terraform

Python

Slack Webhooks